Chapter 1

Getting Started with Client-Side Rails

Every modern web application uses client-side features in an effort to improve the end-user experience. Even though Rails is a server-side framework, it has always offered tools that made it easier to deliver client-side code to the browser. From the very beginning of Rails, its ability to make client-side coding easier has been a selling point.

Over time the client-side tools have become more powerful, which is great, and more complicated, which is less great. One thing that hasn't changed, though, is that Rails still has opinions about how to deal with the client tools.

In this book, we're going to take a basic Rails application and add client-side interactivity. The JavaScript ecosystem includes a lot of tools that allow you to work with Rails; for our purposes, we're going to focus on two patterns that work well with Rails. In one pattern, the server communicates with the client by sending HTML, and the client's job is mostly to direct that HTML to the correct part of the page and add some client-only interactions. In the other pattern, the server communicates by sending data (usually in JSON format), and the client is responsible for the logic to convert that data into HTML and is responsible for client-only interactions.

To demonstrate the first pattern, we'll use the Hotwire family of tools, which comes from Basecamp and is a more-or-less official add-on to Rails.[18] Hotwire consists of Turbo,[19] a library that manages user navigation and communication with the server without needing custom JavaScript, and Stimulus,[20] which supports client-side interactions written in JavaScript. Stimulus is designed to be